A Preliminary Study of Grouping Strategies for ListT5 Reranking

Mariano Gerardus Senduk*
Universitas Indonesia
mariano.gerardus@ui.ac.id

Narendra Dzulqarnain*
Universitas Indonesia
narendra.dzulqarnain@ui.ac.id

Abstract

This paper presents a preliminary inference-only study of ListT5 reranking. We examine two practical choices in the reranking pipeline: the BM25 top-k candidate budget and the first-round grouping policy used before ListT5 tournament reranking. Using ListT5-base without additional training, we evaluate 50-query subsets of NFCorpus, SciFact, ArguAna, SCIDOCS, and FiQA-2018 with a fixed 512-token input cap. In the current results, sequential top-100 is not consistently the strongest setting; smaller BM25 candidate pools often match or improve NDCG@10 while reducing runtime. The score-balanced top-100 runs do not show a clear quality gain over sequential grouping, although strategy-aware caching substantially reduces their runtime.

1 Introduction

ListT5 is a listwise reranker that uses Fusion-in-Decoder (FiD) with a tournament-style inference procedure to rerank candidates from a first-stage retriever (Yoon et al., 2024). The original ListT5 work studies model design, training, efficiency, and retrieval performance. This paper takes a smaller follow-up direction: we keep the released ListT5-base model fixed and examine two inference-time choices in the reranking pipeline.

The first choice is the BM25 candidate budget. The standard ListT5 evaluation reranks BM25 top-100 candidates, but smaller candidate pools can reduce the number of listwise comparisons and may be more practical in limited compute settings. We therefore compare sequential ListT5 reranking with BM25 top-40, top-60, top-80, and top-100 candidates. In this paper, sequential top-100 is the local baseline because all runs use the same 50-query subset.

The second choice is the first-round grouping policy. In the original implementation, candidates are grouped sequentially by BM25 rank. With listwise group size five, candidates ranked 1–5 are compared together, candidates ranked 6–10 are compared together, and so on. We explore a score-balanced alternative that distributes BM25 ranks across groups, such as grouping ranks 1, 21, 41, 61, and 81 together when reranking top-100 candidates. This does not change the model or candidate set; it only changes how candidates are arranged before the tournament comparisons.

The experiment is intentionally lightweight. We use the public ListT5-base checkpoint, evaluate the first 50 queries per dataset, cap inputs at 512 tokens, and report both NDCG@10 and runtime. The current manuscript is structured around two questions:

1. How does reducing the BM25 candidate budget from top-100 to top-80, top-60, or top-40 affect sequential ListT5 reranking?

2. Under the same 50-query setup, how does score-balanced grouping compare with sequential grouping at BM25 top-100?

2 Related Work

Listwise reranking processes multiple candidate passages together so the model can compare candidates within the same inference call. ListT5 implements this idea with an FiD architecture, where multiple query–passage inputs are encoded separately and decoded into an ordered list of passage identifiers (Izacard and Grave, 2021; Yoon et al., 2024). To make this practical for reranking a larger candidate pool, ListT5 uses an m-ary tournament sort with output caching.

Our study is a small extension of this inference pipeline. We do not modify the ListT5 architecture, retrain the model, or introduce a new reranking objective. Instead, we study two practical inference choices: how many BM25 candidates should be

*These authors contributed equally to this work.

passed to ListT5, and how those candidates should be grouped before the first tournament round.

3 Method

Let $C = [c_1, c_2, \ldots, c_k]$ be the candidate list returned by BM25, sorted from highest to lowest BM25 score. ListT5 reranks this list by repeatedly applying a listwise model to groups of size m. In our experiments, $m = 5$ and the model outputs $r = 2$ selected candidates per listwise call, matching the released ListT5-base setup.

3.1 BM25 Top-k Setting

The first experiment changes the number of BM25 candidates passed to ListT5. We evaluate sequential ListT5 reranking with $k \in \{40, 60, 80, 100\}$. This directly tests the quality–runtime tradeoff of reducing the first-stage candidate pool while leaving the ListT5 model and grouping policy unchanged.

3.2 Sequential Grouping

The baseline grouping policy is the original ListT5 behavior. Candidates are split into contiguous chunks:

$$G_i = [c_{(i-1)m+1}, \ldots, c_{im}]. \tag{1}$$

For BM25 top-100 with $m = 5$, this creates 20 first-round groups. This policy keeps neighboring BM25 ranks together and is used as the main comparison point throughout the experiment.

3.3 Score-Balanced Grouping

The proposed tweak keeps the same candidate list and the same group size, but changes how the first-round groups are formed. Let $n_g = \lceil k/m \rceil$ be the number of groups. Candidates are assigned in round-robin order by BM25 rank:

$$c_j \rightarrow G_{((j-1) \bmod n_g)+1}. \tag{2}$$

For BM25 top-100 and $m = 5$, the first group receives ranks 1, 21, 41, 61, and 81; the second group receives ranks 2, 22, 42, 62, and 82; and so on. This distributes the BM25 ranking spectrum across groups, so each first-round comparison contains one high-ranked candidate and several lower-ranked candidates.

Dataset	Code name
NFCorpus	`nfcorpus`
SciFact	`scifact`
ArguAna	`arguana`
SCIDOCS	`scidocs`
FiQA-2018	`fiqa`

Table 1: Datasets used in the 50-query early experiment.

Strategy-aware first-round caching. The original ListT5 implementation precomputes first-round cache entries for sequential chunks such as ranks 1–5 and 6–10. This cache is aligned with sequential grouping, but it does not match score-balanced groups. In the v2 notebook, the cache precomputation is updated to generate first-round cache keys from the active grouping policy. This keeps the score-balanced experiment comparable to sequential grouping in runtime measurement, because both policies can use cached first-round model outputs for their actual initial groups.

4 Experimental Setup

4.1 Datasets

We use five BEIR datasets (Thakur et al., 2021): NFCorpus, SciFact, ArguAna, SCIDOCS, and FiQA-2018. These datasets cover scientific, biomedical, argument retrieval, citation-related, and financial question-answering retrieval settings. The current notebook uses the first 50 queries from each dataset to keep the early experiment practical on Kaggle. Table 1 lists the dataset identifiers used by the code.

4.2 Model and Inference Settings

All experiments use the public `Soyoung97/ListT5-base` checkpoint. The input candidates are BM25 top-100 files from the BEIR-format dataset repository used by the ListT5 codebase. The fixed ListT5 settings are listwise group size $m = 5$, output size $r = 2$, reranking depth 10, maximum input sequence length 512, and NDCG@10 evaluation. The 512-token cap is applied to every dataset to keep the early run consistent and practical in the low-resource notebook setting. For the BM25 top-k experiment, the input top-k value is changed to 40, 60, or 80 and compared with sequential top-100 as the local baseline.

4.3 Compared Conditions

The experiment grid has two parts. First, the BM25 top-k comparison evaluates sequential grouping at BM25 top-40, top-60, top-80, and top-100. Second, the grouping comparison evaluates sequential and score-balanced grouping at BM25 top-100. Sequential top-100 is reused as the local baseline in both comparisons, so the full grid has five jobs per dataset: sequential top-40, top-60, top-80, top-100, and score-balanced top-100. The reported runs were mainly executed from notebooks for Kaggle-style experimentation. The repository also contains a Python codebase version of the grouping experiment, which can be used when moving the workflow from interactive runs to scripted execution.

5 Results

Because the current setup uses only 50 queries per dataset, the comparison should be read as an early signal rather than a full benchmark result. The local baseline is sequential ListT5 with BM25 top-100 under the same 50-query setting. Table 2 combines the BM25 top-k and grouping comparisons. The sequential columns compare BM25 top-k budgets, while the score-balanced column reports the aligned-cache v2 grouping result at BM25 top-100.

Runtime is reported as average seconds per query in Table 3. This is more compact than total runtime and stays comparable across datasets and conditions. The score-balanced columns separate the original sequential-cache behavior from the v2 strategy-aware cache.

Table 4 reports the number of ListT5 forward calls for the first 50 queries. This gives a model-level cost view that is less dependent on runtime noise from the notebook environment.

6 Analysis

BM25 top-k tradeoff. The current results show that sequential top-100 is not consistently the strongest setting on the 50-query subsets. Top-80 gives the highest NDCG@10 on SciFact, SCIDOCS, and FiQA-2018, while top-40 is highest on ArguAna and top-60 is slightly highest on NFCorpus. This suggests that increasing the BM25 candidate pool to 100 does not automatically improve the early ListT5 reranking result under this small-query setup. For runtime and forward-call count, top-60 is currently the cheapest setting on all five datasets, while top-80 and top-100 are slower and require more model calls.

Grouping effect. The aligned-cache score-balanced top-100 results are available for all five datasets. Score-balanced grouping is below sequential top-100 on NFCorpus, SciFact, ArguAna, and SCIDOCS, and nearly tied with sequential top-100 on FiQA-2018. Overall, the current results do not show a clear quality advantage for score-balanced grouping.

Dataset variation. The best BM25 top-k setting differs by dataset, so we avoid relying only on an average score. For example, ArguAna favors a much smaller top-40 candidate set in the current table, while SCIDOCS and FiQA-2018 favor top-80. This variation is consistent with the paper's framing as a preliminary diagnostic study rather than a final benchmark result.

Caching effect. The strategy-aware cache substantially reduces score-balanced runtime across all five datasets. Aligned caching is roughly two times faster than using the original sequential-cache behavior. The forward-call counts show the same trend: aligned caching reduces score-balanced model calls by about two times compared with the original sequential-cache behavior. However, score-balanced top-100 remains slower than sequential top-100 and much slower than the fastest sequential top-k setting in the current runs.

7 Future Work

The immediate next step is to replace the 50-query subset with full-query evaluation on the same five datasets. This would make the comparison less sensitive to query sampling and would support stronger conclusions about the BM25 top-k tradeoff.

The score-balanced setting should also be expanded beyond BM25 top-100. In the current experiment, smaller top-k values are tested only with sequential grouping, so future runs should evaluate score-balanced grouping at top-40, top-60, and top-80 as well. This would separate the effect of grouping from the effect of candidate-pool size.

Another direction is adaptive top-k selection. Instead of choosing the same BM25 top-k for every query, the reranking system could select a candidate budget based on query difficulty, BM25 score distribution, or early tournament confidence. This may preserve quality for difficult queries while reducing runtime for easier ones.

	Sequential grouping				Score-balanced
Dataset	Top-40	Top-60	Top-80	Top-100	Top-100
NFCorpus	0.33718	**0.34292**	0.34212	0.34274	0.33855
SciFact	0.72975	0.72829	**0.73023**	0.72683	0.72262
ArguAna	**0.46052**	0.44007	0.41867	0.43436	0.41691
SCIDOCS	0.16004	0.15644	**0.17279**	0.16678	0.16164
FiQA-2018	0.41760	0.43894	**0.45958**	0.45442	0.45446

Table 2: NDCG@10 on the first 50 queries of each dataset. Bold marks the highest score in each row.

	Sequential grouping				Score-balanced top-100	
Dataset	Top-40	Top-60	Top-80	Top-100	Seq. cache	Aligned cache
NFCorpus	17.55	**16.49**	22.61	24.88	85.54	40.89
SciFact	23.48	**22.04**	30.52	33.65	119.90	58.40
ArguAna	22.67	**21.15**	29.61	32.27	118.74	60.98
SCIDOCS	22.26	**20.97**	29.05	31.68	118.71	57.03
FiQA-2018	23.43	**22.30**	30.74	33.73	128.86	56.71

Table 3: Average runtime per query in seconds. Bold marks the fastest setting in each row. Lower is better.

8 Conclusion

We presented a small inference-only study of ListT5-base under a 50-query evaluation setup. The current results suggest that reducing BM25 top-k can be competitive with sequential top-100: the best NDCG@10 setting varies by dataset, and top-60 is the fastest setting across all five datasets. For the grouping experiment, aligned-cache score-balanced top-100 does not show a clear quality improvement over sequential top-100. The caching update is still useful, reducing score-balanced runtime by roughly a factor of two. Overall, the strongest immediate signal is that BM25 top-k deserves careful tuning before scaling the grouping comparison to full-query evaluation.

Limitations

This is a preliminary experiment and uses only the first 50 queries from each dataset. The results should not be reported as a reproduction of the full BEIR benchmark. The subset may overrepresent or underrepresent difficult queries, and the variance may be large.

The study changes only inference-time grouping. It does not retrain ListT5, tune hyperparameters, or test whether the model would learn different behavior if trained with score-balanced groups. The BM25 top-k experiment is also limited to sequential grouping, so it should not be interpreted as a complete search over grouping and top-k combinations. All datasets are also evaluated with a fixed maximum input sequence length of 512 tokens. This keeps the run controlled, but it may truncate inputs for datasets that could benefit from longer context.

Finally, runtime measurements can depend on GPU type, batch size, caching, and notebook environment. Runtime comparisons should therefore be reported with the hardware and batch size used in the actual run.

Ethics Statement

This work evaluates retrieval reranking methods on existing BEIR benchmark datasets and does not introduce new human-subject data collection. The main ethical considerations are inherited from the datasets and from the downstream use of retrieval systems. Rerankers can affect which information users see first, so performance differences should be interpreted carefully across domains and should not be assumed to imply fairness, factuality, or reliability in deployment. Any released results should cite the original datasets and model sources and respect their licenses.

	Sequential grouping				Score-balanced top-100	
Dataset	Top-40	Top-60	Top-80	Top-100	Seq. cache	Aligned cache
NFCorpus	2267	**2011**	2780	2969	11787	5609
SciFact	2955	**2616**	3646	3936	16628	7635
ArguAna	3003	**2628**	3717	3963	16563	7729
SCIDOCS	2970	**2637**	3655	3934	16628	7735
FiQA-2018	2895	**2591**	3592	3865	16827	7535

Table 4: Number of ListT5 forward calls for the first 50 queries. Bold marks the fewest calls in each row. Lower is better.

References

Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*, pages 874–880.

Nandan Thakur, Nils Reimers, Andreas Rücklé, Abhishek Srivastava, and Iryna Gurevych. 2021. BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*.

Soyoung Yoon, Eunbi Choi, Jiyeon Kim, Hyeongu Yun, Yireun Kim, and Seung-won Hwang. 2024. Listt5: Listwise reranking with fusion-in-decoder improves zero-shot retrieval. *arXiv preprint arXiv:2402.15838*.

A Appendix

A.1 Reproducibility

The experiments are implemented in the ListT5 project notebooks. The v2 notebook contains the strategy-aware caching update used for the score-balanced aligned-cache runs.

The key configuration for the preliminary run is:

- model: `Soyoung97/ListT5-base`
- datasets: `nfcorpus`, `scifact`, `arguana`, `scidocs`, `fiqa`
- maximum queries per dataset: 50
- maximum input sequence length: 512
- listwise group size: 5
- output size per listwise call: 2
- reranking depth: 10
- BM25 top-k comparison: sequential top-40, top-60, top-80, and top-100
- grouping comparison: sequential top-100 vs score-balanced top-100
- v2 caching check: strategy-aware first-round cache for score-balanced grouping

A.2 Grouping Example

With 20 candidates and group size 5, sequential grouping forms:

`[1,2,3,4,5]`, `[6,7,8,9,10]`, `[11,12,13,14,15]`, `[16,17,18,19,20]`.

Score-balanced grouping forms:

`[1,5,9,13,17]`, `[2,6,10,14,18]`, `[3,7,11,15,19]`, `[4,8,12,16,20]`.

The implementation uses zero-based candidate indices internally, but the paper describes ranks using one-based indexing for readability.